

AI IN SOFTWARE DEVELOPMENT

Dr. Ehab Essa
Computer Science Department,
Mansoura University

AI IN SOFTWARE DEVELOPMENT

Artificial intelligence (AI) is revolutionizing the software development process by introducing tools and techniques that enhance productivity, accuracy and innovation. From automating code generation to optimizing testing and deployment, AI is reshaping how software is designed, built and maintained.

AI, particularly **generative AI (gen AI)** and **large language models (LLMs)**, streamline the development cycle by automating key steps, from idea generation and requirement gathering to coding and testing.

AI has a significant impact on **code generation**. Machine learning-enabled tools use **natural language processing (NLP)** to interpret natural language descriptions and produce code suggestions or complete code. This capability accelerates coding, reduces human error and allows developers to focus on more complex and creative tasks rather than boilerplate code.

AI AND GEN AI

Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.

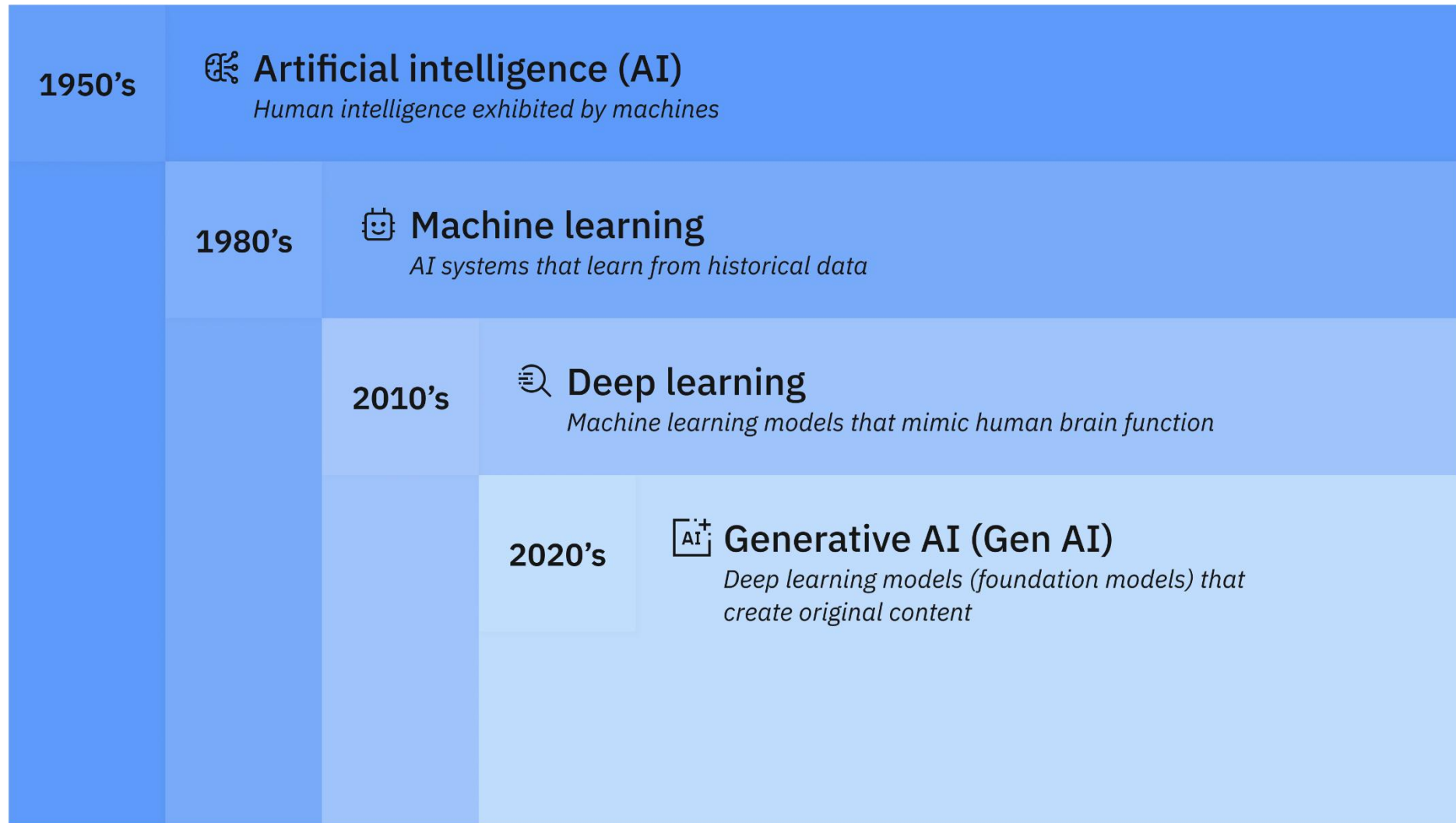
Generative AI, sometimes called **gen AI**, is artificial intelligence (AI) that can create original content such as text, images, video, audio or software code in response to a user's prompt or request.

- Generative AI relies on sophisticated machine learning models called deep learning models algorithms.
- These models work by identifying and encoding the patterns and relationships in huge amounts of data, and then using that information to understand users' natural language requests or questions and respond with relevant new content.

AI has been a hot technology topic for the past decade, but generative AI, and specifically the arrival of **ChatGPT** in 2022, has thrust AI into worldwide headlines and launched an unprecedented surge of AI innovation and adoption.

Generative AI uses **large language models (LLMs)** for creating text generation applications.

HOW ARTIFICIAL INTELLIGENCE, MACHINE LEARNING, DEEP LEARNING AND GENERATIVE AI ARE RELATED.



AI IN SOFTWARE DEVELOPMENT

AI-powered **autocompletion** and **code synthesis** further improve productivity by predicting the next lines of code or even generating entire functions.

Beyond coding, AI technologies enhance **debugging** and **testing**. Advanced AI tools can automatically detect bugs, vulnerabilities and inefficiencies and suggest fixes or optimizations. AI-driven testing systems generate adaptive test cases and prioritize the most critical tests, improving software quality and security.

AI helps developers avoid future issues with its ability to **predict errors** based on historical data.

Overall, AI is increasing **development speed** and **accuracy** and fostering a more **reliable** and secure software environment.

The future will bring even more advancements. As **gen AI** evolves, it might fundamentally reshape every stage of development.

HOW AI IS USED IN SOFTWARE DEVELOPMENT

Key areas where AI is used in software development include:

- Code generation
- Bug detection and fixing
- Testing automation
- Documentation
- Refactoring and optimization
- Security enhancement
- Project management
- UX design
- Architecture design

CODE GENERATION

AI-powered tools assist developers by suggesting code or generating entire functions from natural language inputs, speeding up development by automating routine tasks.

Tools such as Cursor, Amazon CodeWhisperer and GitHub Copilot help developers write code faster and with fewer errors and can generate suggestions and autocomplete code.

Autocompletion:

- AI predicts and suggests the next lines of code, improving speed and reducing errors.
- Example: Writing a for loop, the AI may automatically suggest the loop body and closing statement.

Code synthesis:

- AI automatically writes code for you based on what you describe in words.
- Boilerplate code is code that you must write again and again in almost every project, even though it almost never changes.
- Complete functions
- Sometimes even full classes or modules

CODE GENERATION

Example: Code Generation with AI

Prompt (Natural Language Input):

"Write a Python function that calculates the factorial of a number."

AI-Generated Code (Result):

python

 Copy code

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
# Example usage:  
print(factorial(5)) # Output: 120
```


BUG DETECTION AND FIXING

Gen AI-driven tools can automatically detect bugs, vulnerabilities or inefficiencies in the code. They analyze patterns within the codebase and offer solutions.

- **Error prediction:**

- Predicts where future bugs may appear (e.g., memory leaks, security flaws).
- Example: “This loop could cause an index out-of-range error.”

- **Automated debugging:**

- AI suggests or autocorrects code issues by using real-time data to refine prototypes.
- Some tools autocorrect minor issues (e.g., missing semicolons, unused variables).

EXAMPLE: ERROR PREDICTION & DEBUGGING

Developer Prompt (to ChatGPT): "I get an IndexError in this Python code. Can you fix it?"

python

 Copy code

```
nums = [10, 20, 30]
for i in range(len(nums)):
    print(nums[i+1])
```

ChatGPT Suggestion:

python

 Copy code

```
nums = [10, 20, 30]
for i in range(len(nums) - 1):    #  Prevents index out of range
    print(nums[i+1])
```

TESTING AUTOMATION

AI tools can automatically generate, run, and optimize **tests**. They use **user stories, requirements**, or code changes to create meaningful test cases. **Reduces manual effort** while increasing coverage and reliability.

Test case generation:

- AI reads **requirements / user stories** and generates test scenarios automatically. AI covers more scenarios than manual testing.
- **User Story:** *"As a user, I want to log in with my email and password so that I can access my account."*
- **AI-Generated Test Cases:**
 -  Login with valid email & correct password → Success.
 -  Login with valid email & wrong password → Error message.
 -  Login with invalid email format → Error message.
 -  Login with empty fields → Validation error.
 -  Multiple failed attempts → Account locked.

Test optimization:

- AI prioritizes critical tests to save time and resources. Running **all tests every time** is slow and expensive. AI **chooses the most important tests** to run first, based on **risk, code changes, or priority**.

EXAMPLE: DIVIDE FUNCTION

Code (Developer's Initial Function)

```
python  
  
def divide(a, b):  
    return a / b
```

Prompt to AI:

"Fix this function so it doesn't crash when dividing by zero."

AI Suggestion:

```
python  
  
def divide(a, b):  
    if b == 0:  
        return "Error: Division by zero"  
    return a / b
```

Prompt to AI:

"Generate test cases for this divide function."

Test Results:

Test Case	Expected Result	Status
divide(10, 2)	5.0	✓ Pass
divide(7, -1)	-7.0	✓ Pass
divide(0, 5)	0.0	✓ Pass
divide(10, 0)	"Error: Division by zero"	✓ Pass
divide(-10, 0)	"Error: Division by zero"	✓ Pass

DOCUMENTATION

Gen AI tools use **NLP** to generate and maintain documentation, turning code into readable explanations and helping ensure up-to-date project information. Saves time → no more manual doc writing.

Auto-documentation:

- AI scans code, APIs, or libraries → generates clear documentation.
- Keeps docs in sync with code changes (avoiding outdated manuals).

Translation: AI localizes technical documents into multiple languages, making open-source projects more accessible globally.

Summarization:

- AI converts long technical specifications → short summaries.
- Makes documentation more readable for non-technical stakeholders.

EXAMPLE: DOCUMENTATION

Code

```
python

def factorial(n):
    return 1 if n == 0 else n * factorial(n - 1)
```

AI-Generated Documentation:

Function: factorial(n)

Description:

Computes the factorial of a number n using recursion.

Parameters:

n (int): A non-negative integer.

Returns:

int: The factorial of n.

Examples:

factorial(5) → 120

REFACTORING AND OPTIMIZATION

Generative AI tools help developers **improve the quality of existing code** by suggesting better structure, readability, and performance improvements.

This makes software easier to maintain, more efficient, and aligned with **best practices**.

Code review: AI detects bad practices (check code quality) and suggests improvements based on computer science best practices.

- e.g. Unused variables or imports. Duplicate code. Functions that are too long. Poorly named variables/functions.
- Following language/framework conventions. Consistent formatting & indentation.

Performance optimization: AI analyzes and improves code efficiency. It's the process of making code run faster, use less memory, and scale better while keeping the same functionality.

- e.g. Detects performance bottlenecks (slow DB queries, inefficient loops, memory leaks).

EXAMPLE: CODE REVIEW

Prompt to AI:

"Review this function and suggest improvements."

```
python

def calculate_total(prices):
    total = 0
    unused = 5    # not used
    for p in prices:
        total += p
    return total
```



AI Suggestion (Result):

```
python

def calculate_total(prices):
    total = 0
    for p in prices:
        total += p
    return total
```

Prompt to AI:

"Suggest improvements based on best practices."

```
python

def process(data):
    for i in range(len(data)):
        print(data[i])
```



```
python

def process(data):
    for item in data:
        print(item)
```


EXAMPLE: CODE OPTIMIZATION

Inefficient Loop

Developer Prompt:

"Can you optimize this code?"

python

 Copy code

```
result = []  
for i in range(len(data)):  
    result.append(data[i] * 2)
```

ChatGPT Suggestion:

python

 Copy code

```
# Use list comprehension for efficiency  
result = [x * 2 for x in data]
```

SECURITY ENHANCEMENT

AI-driven tools identify vulnerabilities, monitor code for security threats and help fix security risks in code

Threat detection:

- Identifying potential or active attacks that target your application, database, or users.
- AI tools analyze code, logs, traffic, and user input to find suspicious patterns that might indicate SQL Injection (SQLi), Cross-Site Scripting (XSS), brute force, or other attacks.

Code auditing:

- Systematically reviewing source code to find **security flaws, bad practices, or non-compliance** with secure coding standards.
- Every time a developer makes a code change (commit or pull request), AI scans it.
- AI ensures new code follows **secure coding guidelines**

EXAMPLE: SQL INJECTION

SQL Injection is a security vulnerability where an attacker inserts malicious SQL code into a query.

This happens when user input is directly concatenated into SQL strings without validation or **parameterization**.

If not prevented, attackers can: Read sensitive data (like passwords, credit card info). Modify or delete database records. Take full control of the database.

Prompt to AI: “Is this safe? If not, fix it.”

```
python

username = request.args.get("u")
query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)
```



AI Suggestion (fix):

```
python

username = request.args.get("u")
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

AI'S EFFECT ON THE SOFTWARE DEVELOPMENT LIFECYCLE (SDLC)

Generative AI is transforming the SDLC by automating processes, accelerating development time, improving code quality and reducing costs.

Generative AI can enhance productivity and optimize efficiency at each stage. Here's how gen AI is impacting the SDLC:

- Requirement gathering and analysis
- Design and planning
- Development
- Testing
- Deployment
- Maintenance and support
- Documentation

REQUIREMENT GATHERING AND ANALYSIS

It's the first phase of SDLC where stakeholders' needs are collected, clarified, and transformed into clear, testable requirements. Traditional way: interviews, workshops, surveys, documents.

Gen AI converts high-level ideas into detailed requirements by processing natural language inputs. It analyzes business goals and user needs to propose features or anticipate requirements, speeding up this phase and reducing errors.

Requirement Extraction

- AI reads documents, emails, meeting transcripts, or chats to extract potential requirements.
- NLP identifies phrases like “the system should...” or “users need to...”.

Requirement Clarification

- AI spots ambiguous requirements like: “System must load fast” → What does fast mean?
- AI suggests clarification: “System must load within 2 seconds under 1,000 concurrent users.”

Requirement Classification

- AI organizes requirements into categories: Functional (what system does). Non-functional (performance, security usability). Constraints (compliance, cost, platforms).
- Example: Functional: “Allow login with Google account.” Non-functional: “System must handle 10,000 users per second.”

DESIGN AND PLANNING

The Design & Planning phase is where developers, architects, and stakeholders decide:

- How the system will be structured (architecture).
- How it will look and feel (UI/UX).
- What technical components will be used (databases, APIs, services).

Generative AI enhances this stage by providing blueprints, diagrams, mockups, and reusable architectures, reducing manual work and ensuring consistency.

AI suggests **optimal architectures** based on constraints like scalability, performance, or security.

AI converts text descriptions into wireframes and mockups. e.g. **Figma AI**

- Example Prompt: “Generate a mobile login screen with email, password, and social login.”

AI generates UML, ER diagrams, flowcharts, sequence diagrams from natural language.

- Example Prompt: “Create a class diagram for a hospital management system.”

DEVELOPMENT & TESTING

The **Development phase** where actual **coding and implementation** happen.

- Traditionally → developers write code line by line.
- **AI assistants** accelerate coding, debugging, refactoring, and security checks, enabling developers to focus on complex problems while AI-driven autocompletion and real-time suggestions improve both speed and accuracy.
- The **Testing phase** ensures that software meets requirements, works correctly, and is secure. Generative AI improves this phase by **automating test case creation**, analyzing code for areas that need testing. It optimizes coverage, detects bugs early and reduces manual testing time, improving software quality and testing efficiency.

MAINTENANCE & SUPPORT

Maintenance & Support is the longest phase of software's life — after deployment, when the system is running in production. **Main goals:** Fix bugs, Improve performance, Patch security vulnerabilities, Support users, Ensure long-term stability.

Gen AI helps identify areas for code refactoring and optimization post deployment. It continuously monitors performance, detects anomalies and predicts issues, improving reliability and reducing incident resolution time.

- AI analyzes logs, crash reports, and user feedback to detect hidden issues. Suggests or even auto-generates fixes.
- AI tools scan system metrics, logs, and network traffic for anomalies. Detects suspicious behavior like DDoS attacks, memory leaks, or unusual latency.
- AI chatbots handle Level 1 support tickets (password resets, common errors).
- AI suggests performance improvements for legacy code.

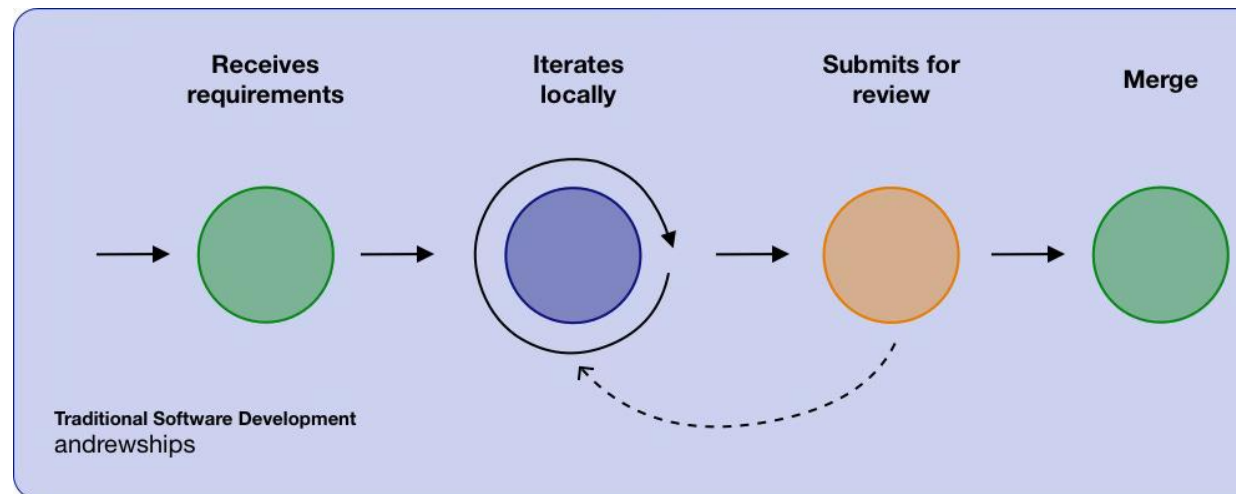
PROMPT DRIVEN DEVELOPMENT (PDD)

PDD is new paradigm in software engineering in which the developers use natural language prompting an LLM to generate all the necessary code instead of writing all code manually.

- The prompt becomes the specification, and
- The AI (e.g., LLMs like GPT-4/5, Copilot, CodeWhisperer) generates code, tests, documentation, or designs.

At a very high level, traditional software development typically looks something like this:

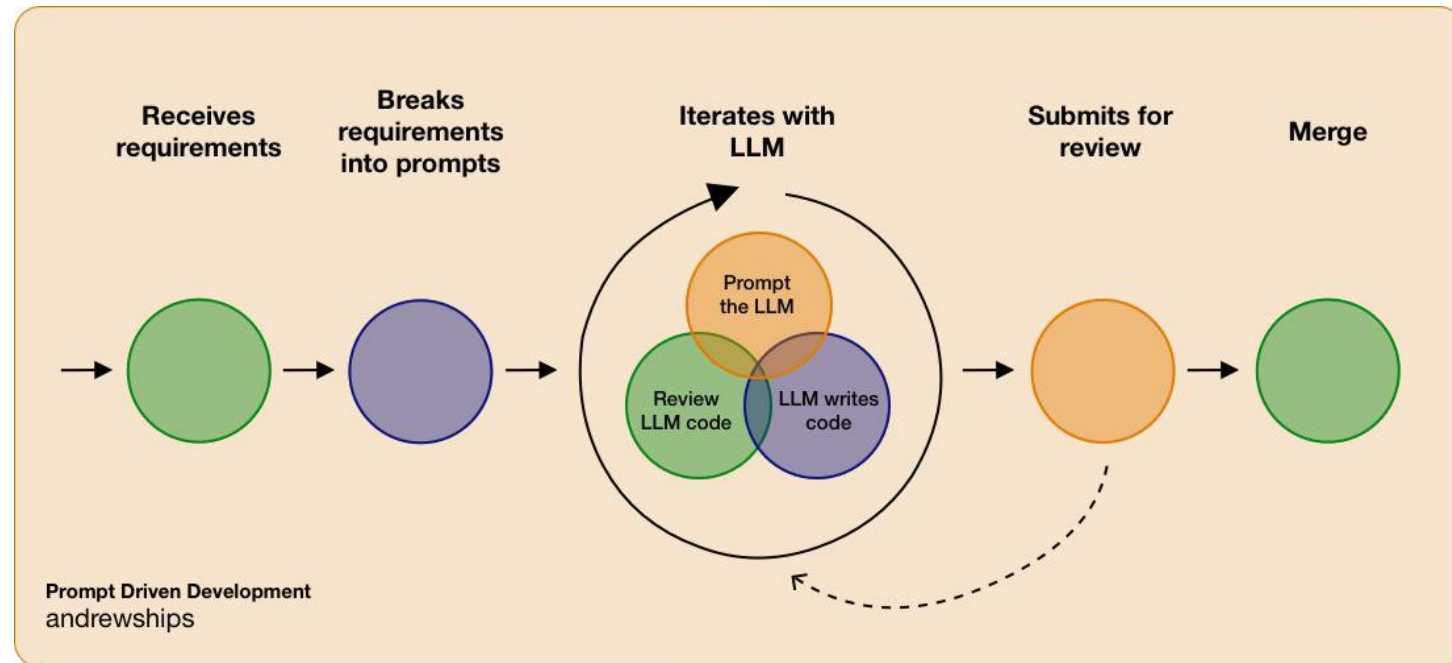
- Developer receives requirements. Developer iterates on code changes locally in their IDE
- Developer submits code changes for review. Another developer reviews and merges the changes



PROMPT DRIVEN DEVELOPMENT (PDD)

High level, prompt driven development flow:

- Developer receives requirements
- Developer breaks requirements down into a series of prompts*
- LLM generates code for each prompt
- Developer reviews LLM generated code*
- Developer submits all changes for review
- Another developer reviews and merges changes



TOOLS

Chats

ChatGPT, Claude, Gemini

It's a bit more manual work to provide all the right context to the LLM, and then to paste the output into your codebase.

The workflow would look like this:

- Write your prompt for the LLM
- Copy all the necessary code context from your codebase and include it as part of the prompt
- Copy the code generated by the LLM
- Paste the code output into the right place in your codebase

use Claude or ChatGPT **Projects**. These features allow you to organize files and chats together, which will help tremendously while providing code context to the LLM.

TOOLS

LLM-Native IDEs

Cursor, Windsurf, Bolt, v0, Replit

They are IDEs that give you access to LLMs within the window of the IDE. They have two huge advantages over using a Chat to generate code:

- It's super easy to provide code context to the LLM
- The LLM is able to edit your files directly in the IDE

This makes for a significantly faster feedback loop while generating code because you don't have to copy+paste code context into a Chat, and then copy+paste the output into your IDE. Those steps just disappear.

I strongly recommend picking an LLM-Native IDE and sticking with it and focus on your prompting.

SUCCESS PROMPTING

Better prompting == better LLM generated code. Garbage in, garbage out, right?

The LLM is a junior dev who's new to the project

When you're crafting your prompt, think about how you might instruct this type of individual. You wouldn't say something like, "build me an app that lets friends post pictures and follow each other."

Instead, you would give them a very small, very specific change to make. And you would probably point them to all the right files, as well as tell them a whole bunch about what the product is, and how people use it.

Context, Specificity, and Scope

These are the three variables you want to be considering as you tune your own prompting. To improve the quality of the LLM generated code, you need to:

- Increase the amount of context you provide
- Increase the specificity of your request
- Narrow the scope of the change

EXAMPLE

Context is the snippets or files from your codebase you provide in your prompt.

- Low context: "Make these changes on the back-end"
- High context: "Adjust the getMarker function in the backend/routes/user.js file. And here is that file, plus 3 more files it commonly interacts with..."

Specificity is how clearly defined and precise your ask is.

- Low specificity: "When a user clicks submit, add their submission to the feed."
- High specificity: "When a user clicks submit, call the saveSubmission endpoint to store the submission in the database. Once saved, automatically call the loadFeed function to refresh the feed on the frontend so that the user is able to see their submission immediately, without having to reload the page."

Scope is how much we are asking the LLM to change at once.

- Broad scope: "Add a settings page where the user can manage and update their account."
- Narrow scope: "Add a toggle on the settings page where the user can set their account to private or public."

LLMS WRITE BUGS (JUST LIKE HUMANS)

Guess what, LLMs aren't perfect. I would recommend curbing your expectations, and just know that the LLM will generate a bug here and there. That is part of the normal development process, even when humans are writing the code.

This is also why it's so important to **review and test the LLM's code** before accepting it. Don't blindly merge the changes it makes.

PROMPT CHAINING

Prompt chaining represents a powerful paradigm for handling intricate tasks when leveraging large language models (LLMs).

Instead of solving a complex task in **one big prompt**, break it into smaller, sequential prompts (**divide-and-conquer strategy**).

Each sub-problem is addressed individually through a specifically designed prompt, and the output generated from one prompt is strategically fed as input into the subsequent prompt in the chain.

Benefits: modularity, clarity, easier debugging, more reliable results.

Why Not a Single Prompt? One large prompt often causes:

- Instruction neglect (parts ignored).
- Contextual drift (losing track of initial request).
- Error propagation (early mistakes amplify).
- Hallucinations (wrong info due to overload).

EXAMPLE

Single Prompt Attempt (Failure)

Prompt:

"Write a Python program that reads a CSV file of student grades, calculates the average, finds the top 3 students, handles missing values, writes results to a new file, and include unit tests and documentation."

Solved with Prompt Chaining

Prompt 1: "Write Python code that reads a CSV file of students (name, grade) and calculates the class average."

Prompt 2: "Update the code to ignore rows with missing or invalid grades."

Prompt 3: "Extend the code to return the top 3 students with the highest grades."

Prompt 4: "Modify the code to write the class average and top 3 students into a new CSV file called results.csv."

Prompt 5: "Write pytest unit tests for the functions that calculate the average and find top students."

Prompt 6: "Generate a Markdown README that explains how to use this program, including sample input and output."